Technical report — William

- **Track:** Track 2A — OpenParlData: Extracting Parliamentary Affairs from PDFs into One Common Structure
- **Event:** Online
- **Team:** RedPandas — Christopher Yazigi, Giovanni Yazbeck, Alfredo Pio Yazbeck
- **Demo:** Firefox extension in `src/extension`; API via `make run`

1. Summary

William converts parliamentary documents from federal, cantonal and communal parliaments into one common, versioned structure (`opd-affair/0.1`) with Apertus 1.5 70B. Every value is tied to a page and a verbatim text span, and a value whose evidence cannot be found is removed. The same pipeline runs in three forms:

- a **Firefox extension**: summarises a PDF or a selection in the document's language and links its topics to Swiss news;
- an in-extension **batch page**: many links → JSON + PDF report;
- an **HTTP API** for production use.

On a 14-document sample (3 levels × DE/FR/IT, all from OpenParlData):

- **147 values were verified against the source text**;
- **70 proposed values were rejected** (65 of them had no quote at all);
- **54 classifications are kept but marked `inferred`**.

Headline insight: a common schema works for *identity* fields: number, title, original type term, addressee, authors (core-field coverage 0.78). It does **not** work, from the PDF alone, for *procedure*: state, decisions, history. Only 2 of 13 procedural states were verifiable, and **0 decisions** were found in the documents.

2. Architecture

```
link ─► fetch (PDF via pypdf / pdf.js, HTML, OpenParlData API & file links)
     ─► OPD hints (body_key → level, OPD type & number of the parent affair, used as a cross-check)
     ─► Apertus 70B: JSON with {v, p, q} per field (prompt william-prompts/0.2)
     ─► verifier: quote search on the page (normalised; space-insensitive fallback) → char span
                  dates: day+month+year must be in the quote; numbers must be in the quote
                  vocab fields: legal term or removed; no quote → status "inferred"
                  type_original → fixed type lookup (DE/FR/IT) overrides the model
     ─► opd-affair/0.1 record + quality block (+ raw model output kept for offline re-verification)
     ─► summary (document language) + topics → Google News RSS (CH edition, document language)
```

- `src/extension/shared/{vocab,prompts}.json`: the single source of prompts and vocabularies, read by both the extension and the API.
- `src/extension/shared/core.js` and `src/william/{verify,extract}.py`: the verifier and record assembly, written twice. `tests/parity.mjs` checks that they produce **identical records** on all 13 stored documents (0 differences).
- Optional translation: Apertus translates the summary and key points into DE / FR / IT / RM / EN (`translate_to` on `/v1/extract` and `/v1/summarize`, or `POST /v1/translate`). The original-language summary is always kept next to the translation.
- `src/william/api.py`: FastAPI (`/v1/extract`, `/v1/report`, `/v1/summarize`, `/v1/schema`, `/v1/vocab`). `make run` starts it in Docker.
- The extension's PDF report is written by a 150-line dependency-free PDF writer (`shared/minipdf.js`). Its output was checked by parsing it back with pypdf in strict mode (27 pages, working links).

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-70B` (8B selectable). It is reached through the CSCS OpenAI-compatible endpoint; `LLM_NAME` / `LLM_BASE_URL` / `LLM_API_KEY` follow the judges' convention.
- **How it is used:** inference only (temperature 0). There are two prompts: structured extraction with quotes, and a summary with topics. No other LLM is used anywhere.
- **Where it runs:** hosted endpoint (CSCS inference API). Any OpenAI-compatible endpoint serving Apertus works, e.g. a self-hosted 8B (~20 GB) for on-prem use; nothing else in the pipeline needs a GPU.
- **Cost:** the 14-document extraction run used 44.6k input and 12.8k output tokens.

- **Other models:** none in the pipeline: every summary, extraction and translation is done by Apertus. During development, Claude (Anthropic) was used as a coding assistant and as the **AI checker** in §5(d). It is not an open-weights model and is not part of the submitted system.

4. Data

`data/sample/urls.txt` lists 14 OpenParlData documents: federal (CHE ×4), cantonal (GE ×2, TI, ZH) and communal (Bern 351, Zürich 261, Lausanne 5586 ×2, Lugano 5192 ×2), in German, French and Italian. They are public parliamentary documents fetched at run time. Page texts are stored in `tests/fixtures/` only for the offline tests. There is no personal data beyond the names of elected officials as published by the parliaments.

5. Evaluation

We have no manually checked gold set (a single-day build). We report four measurements instead: three need no gold labels, and one is an AI check of every kept value, labelled as such.

Setup	Metric (14 docs)	Result
Baseline: Apertus 70B output taken as is (no verification)	values whose evidence cannot be found in the document	70 of 248 proposed values (28 %), all shown as facts
Ours: + quote verification, type lookup, inferred flags	values kept that are correct (AI check, §5d)	155 of 178 (87 %); 0 values without evidence presented as extracted

(a) Evidence verification (`make eval`, verifier v2, prompt v0.2).

level	docs	verified values	inferred	rejected	verified share	core-field coverage*
federal	4	36	11	9	0.64	0.71
cantonal	4	37	16	7	0.62	0.88
communal	6	74	27	54	0.48	0.77
all	14 (1 failed)	**147**	54	70	0.54	0.78

* share of {number, title, type, submission date, addressee, state} that are present.

The same per-language breakdown is printed by `make eval`. The weakest cell is communal FR, with 36 rejections. It comes from one Lausanne PDF whose text layer has **no space characters** ("RéponsedelaMunicipalité…").

(b) Ablations. "Verified" counts include OPD metadata values.

run	verified	rejected
prompt v0.1 + verifier v1 (12 docs; 2 of them were the Fedlex-wall files below, swapped out afterwards)	73	112
prompt v0.2 ("q is REQUIRED…") + punctuation-insensitive matching + vocab-without-quote → `inferred` (14 docs)	132	72
+ space-insensitive fallback for broken text layers (same model output, re-verified offline)	147	70

The v0.1 and v0.2 rows differ in documents as well as in prompt, so they show the direction of the change, not a controlled comparison. The last row is controlled: identical model output, verifier change only.

(c) Agreement with OpenParlData's own harmonized type (`python -m william.evaluate crosscheck`): **5/10** where both exist. The disagreements are informative:

- **Italian vocabulary trap.** Ticino's *interrogazione* is a written question (OPD id 12). Apertus labelled it *interpellation*. A fixed term lookup on the verified original word now fixes this. That is +1, and the case is flagged in the record.
- **Document vs affair.** A consultation report inside a parliamentary initiative, a committee report inside a government affair, and a list of proposals in a Bern government affair are all correct *for the document* but not for the affair → schema proposal `document.type` (docs/schema.md §Known gaps).
- **Possible OPD labelling inconsistency.** A Ticino item numbered `I126.19` with the original term *Interpellazione* is harmonized by OPD as *question*.

(d) AI check of every kept value (`eval/ai_check_kept_values.json`). One Claude pass judged each of the 178 values William kept (OPD metadata excluded) against its quote and the document: right value in the right field? **This is not a human gold set.**

status	values	correct	wrong	unsure	precision
extracted	91	82	8	1	0.90
normalised	45	36	7	2	0.80
inferred	42	37	3	2	0.88
all	**178**	**155**	**18**	**5**	**0.87** (0.90 if unsure = right)

The 18 errors are mostly the *wrong field*, not invented values:

- a committee or executive recorded as the parliament (3);
- the addressee of the answer letter, or the court an election is for, recorded as the affair's addressee (2);
- several people merged into one actor (2);
- a report or consultation date taken as the submission date (2);
- type and state errors (5);
- a paraphrased request whose quote does not support it (3).

`normalised` is the weakest status: a paraphrase only needs *some* quote on the page. The fix for v0.2 is to require word overlap between a paraphrase and its quote. One case is worth noting: `type_original` "interpellazione" contradicts its own quote "TESTO DELL'INTERROGAZIONE".

Data-quality findings in OpenParlData itself.

- For at least two federal documents (e.g. doc 485241, "FF 2013 1073"), OPD's stored file and `text` are the Fedlex *"please enable JavaScript"* page, not the PDF. William detects this and flags *"source is listed as a PDF but the file is not a PDF"* instead of extracting from it.
- OPD's `state_name_harmonized_id` is filled for **278 of 326,334 affairs (0.09 %)**, measured on 16 Oct 2026 via the OPD endpoint `group_by/states_harmonized`.

Cross-parliamentary question: *can the procedural state of affairs be read from the documents?* OPD cannot answer this today (0.09 % coverage). Our answer is **mostly no**:

- William proposed a state for 13/13 documents, but only 2 had a verbatim basis; 11 are `inferred`;
- none of the 13 documents contained a parliamentary decision with an outcome.

State and outcome should come from structured event data (OPD `events/votings`), with documents used as evidence, not as the source. This is a well-founded *partial negative* result for the common-schema question.

6. Limitations

- There is no human gold set. Precision of the kept values (0.87) comes from a single AI check (§5d), not from people; recall (values the model missed) is not measured.
- Only the first 28k characters are read; long documents are truncated and flagged. The Zürich 261 document failed in the v0.2 run because of a verifier bug (a character whose lowercase form is two characters). The bug is fixed and tested, but that document was not re-run, to save API cost. With v0.1 it returned no usable values.
- There is no OCR: scanned PDFs are flagged ("little or no text layer"), not processed.
- One PDF = one affair. Bundles of several affairs per PDF are not split.
- News links come from Google News search: related coverage, not verified to be about the same affair. Results are re-ranked by shared content words with the topic, and unrelated items are dropped when related ones exist. Topic phrases

are paraphrased by Apertus; a fuzzy locator (also for phrases copied from space-less PDF text) places them inline. Topics it cannot place are shown as clickable chips.

- The API has no authentication and spends the operator's Apertus credits. It refuses private/local addresses (SSRF guard), caps downloads at 25 MB and sends no CORS headers. Run it behind a gateway in production.
- Tested:
  - **Extension end to end in real (headless) Firefox, live against Apertus: 19/19 checks** (`tests/e2e_firefox.py`, results and screenshots in `eval/e2e/`). Covered: PDF auto-open in the viewer, rendering with a selectable text layer, document and selection summaries in the document language, inline topic highlights → Swiss news, extraction with p.N evidence jumps, JSON download, the batch page with 3 links incl. a dead one, JSON + PDF report downloads, the popup, and the in-page panel on a live parlament.ch page.
  - Live API in Docker (`/v1/extract` with summary + news, `/v1/report`, SSRF refusal, no CORS); offline tests (`make test`, 10 tests); JS/Python parity (0 differences); `web-ext lint` (0 errors).
  - Not covered by automation: the right-click menu itself (Selenium cannot open browser menus; the same code path was driven directly).
- The first Firefox run found 3 bugs, now fixed:
  - one batch document failed on cut-off model JSON (now: one stricter retry with a larger output budget);
  - Apertus returned the language code "ge" (now normalised, with a stopword fallback);
  - an over-specific news query returned nothing (now: progressively shorter queries).
- Google News has no Swiss edition for Italian: `it-CH` is silently redirected to the Italy edition. Italian news queries are therefore restricted to Swiss outlets (RSI, laRegione, CdT, tio, swissinfo, ticinonews).

7. Reproducibility

Submitted state: git tag **`submission`** on `main` of github.com/ChristoYusj/william. `make test` and `make eval` reproduce every number in §5 without an API key. They use the stored Apertus outputs in `eval/sample_70b.json` and the page texts in `tests/fixtures/`. `make sample` re-runs the extraction live. Apertus at temperature 0 is not perfectly deterministic, so expect small differences.

8. Next steps

- A 60–100-affair gold set (3 languages × 3 levels), checked by people, to measure slot precision and recall.
- Schema 0.2: `document.type`, `affairs[]` per document, and state/decisions taken from OPD events.
- Chunked reading for long documents; OCR for scanned PDFs (an open OCR model, on-prem).
- Run with the 8B model on-prem (~20 GB) and compare against 70B on the same verifier.

License

- **Code** (`src/`, `tests/`, Makefile, Dockerfile): Apache License 2.0 (repository `LICENSE`).
- **Report and documentation** (this report, `docs/`, READMEs): Creative Commons Attribution 4.0 (CC-BY-4.0).
- **Data produced by us** (`data/sample/`, `eval/`, `tests/fixtures/`): CDLA-Permissive-2.0. The page texts in `tests/fixtures/` come from official parliamentary documents published by Swiss parliaments, via OpenParlData.
- **Third party:** pdf.js (Apache-2.0, Mozilla) is included in `src/extension/lib/`.

References

- OpenParlData API and harmonized vocabularies: https://api.openparldata.ch/v1 (types / states `group_by` endpoints, accessed 16 Oct 2026).
- Apertus 1.5 (Swiss AI Initiative), served via the CSCS inference API.
- Wikidata QIDs for affair types: listed in `src/extension/shared/vocab.json`.
- Akoma Ntoso (OASIS LegalDocML), Popolo (person/membership roles), JATS (as a design reference for a middle layer).
- pdf.js (Mozilla), pypdf, fpdf2, FastAPI.